



Security Assessment



ether.fi – EigenLayer Deprecation

August 2026

Prepared for ether.fi

Table of contents

Project Summary	3
Project Scope	3
Project Overview	3
Findings Summary	4
Severity Matrix	4
Detailed Findings	5
EigenLayer Deprecation	7
Project Overview	7
Low Severity Issues	8
L-01: Consolidation target is completely unconstrained for pod-less nodes and disabled pods	8
Informational Issues	10
I-01. Pod-less exit requests can be accepted before validator activation	10
I-02. Dropping the `getEigenPod() != address(0)` precondition makes `deployedEtherFiNodes` the sole trust anchor for withdrawal credentials, and `backfillExistingEtherFiNodes` performs no verification	11
I-03. `sweepFunds(address)` performs an unvalidated external call and can emit forged `FundsTransferred` events	12
I-04. Overpaid EIP-7002/7251 fees are never refunded and distort LiquidityPool out-of-LP accounting when swept and are not refunded to the payer	13
I-05. NatSpec blocks orphaned by the new insertions now document the wrong functions, including the credential resolver	14
I-06. `srcPubkey == targetPubkey` consolidations against pod-less nodes are guaranteed consensus-layer no-ops, are exempt from the consolidation rate limiter, and still emit success events	15
I-07. NatSpec on the EL-trigger functions now materially misdescribes behaviour for two of the three credential regimes, including the fee and grouping contracts operators depend on	16
I-08. Exit rate limit uses requested gwei, not actual withdrawal	17
Disclaimer	18
About Certora	18

Project Summary

Project Scope

Project Name	Initial Commit Hash	Latest Commit Hash	Platform	Start Date	End Date
EigenLayer Deprecation	32d0a320de	692d3f75d9	EVM	8/24/2025	8/25/2026

Project Overview

This document describes the manual code review of several modules and changes to the core contracts repository.

The work was a 2 day effort undertaken between **8/24/2025** and **8/25/2025**

The team performed a manual audit of the Solidity smart contracts. During the manual audit, the Certora team discovered bugs in the Solidity smart contracts code, as listed on the following page.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	1	1	1
Informational	8	8	3
Total	9	9	4

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
EigenLayer Deprecation			
L-01	Consolidation target is completely unconstrained for pod-less nodes and disabled pods	Low	Fixed
I-01	Pod-less exit requests can be accepted before validator activation	Low	Acknowledged
I-02	Dropping the <code>`getEigenPod() != address(0)`</code> precondition makes <code>`deployedEtherFiNodes`</code> the sole trust anchor for withdrawal credentials, and <code>`backfillExistingEtherFiNodes`</code> performs no verification	Low	Fixed
I-03	<code>`sweepFunds(address)`</code> performs an unvalidated external call and can emit forged <code>`FundsTransferred`</code> events	Low	Acknowledged
I-04	Overpaid EIP-7002/7251 fees are never refunded and distort LiquidityPool out-of-LP	Low	Acknowledged

	accounting when swept and are not refunded to the payer		
I-05	NatSpec blocks orphaned by the new insertions now document the wrong functions, including the credential resolver	Low	Fixed
I-06	`srcPubkey == targetPubkey` consolidations against pod-less nodes are guaranteed consensus-layer no-ops, are exempt from the consolidation rate limiter, and still emit success events	Low	Acknowledged
I-07	NatSpec on the EL-trigger functions now materially misdescribes behaviour for two of the three credential regimes, including the fee and grouping contracts operators depend on	Low	Fixed
I-08	Exit rate limit uses requested gwei, not actual withdrawal	Low	Acknowledged

EigenLayer Deprecation

Project Overview

This report presents the findings of a manual code review for the **EigenLayer Deprecation** audit within the **EtherFi** core contracts. The work was undertaken between **August 24th** and **August 25th 2026**.

The following contract list is included in the scope of this audit:

- src/oracle/EtherFiAdmin.sol
- src/staking/EtherFiNode.sol
- src/staking/EtherFiNodesManager.sol
- src/staking/StakingManager.sol

The code modifications examined during this review were implemented in the following pull request - [PR#485](#)

Low Severity Issues

L-01: Consolidation target is completely unconstrained for pod-less nodes and disabled pods

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/staking/EtherFiNodesManager.sol	Status: Fixed	

Description:

Before this change, every consolidation request was forwarded to the node's `EigenPod`, and `EigenPod.requestConsolidation` requires *both* the source **and the target** pubkey to be `ACTIVE` validators of that same pod. That EigenLayer-side check was the only thing constraining where consolidated stake could go.

The new pod-less path removes it entirely and nothing replaces it:

- `EtherFiNodesManager.requestConsolidation` validates only the **source** pubkeys, and the code comments make the omission explicit: *"the target is deliberately unconstrained; it may live outside this node"*.
- `EtherFiNode.requestConsolidation` then calls the EIP-7251 predeploy directly with `bytes.concat(srcPubkey, targetPubkey)` — the node is the source validators' withdrawal address, so the request is authorized by the consensus layer.

Consequently, a batch of consolidation requests whose `targetPubkey` is an attacker-operated compounding validator will be accepted, and the entire effective balance of each source validator is permanently transferred to a validator whose withdrawal credentials the protocol does not control. The same issue exists on the `else if (_podRestakingDisabled(target))` branch: once a pod is retired, EigenLayer stops enforcing membership and this contract still validates only sources.



Recommendations:

Require that the target must be a protocol-linked validator resolving to a valid EtherFiNode for all cases where funds move.

JavaScript

```
for (uint256 i = 0; i < requests.length; i++) {
    bytes32 srcHash = calculateValidatorPubkeyHash(requests[i].srcPubkey);
    bytes32 tgtHash = calculateValidatorPubkeyHash(requests[i].targetPubkey);
    if (tgtHash == srcHash) continue; //
switch: moves no value
    if (address(etherFiNodeFromPubkeyHash[tgtHash]) != address(0)) continue; //
known to us
    if (target != address(node) //
pod-backed: provable into the pod
        && IEigenPod(target).validatorStatus(tgtHash) !=
        IEigenPodTypes.VALIDATOR_STATUS.INACTIVE) continue;
    revert UnknownConsolidationTarget();
```

We also recommend splitting the `EXECUTOR_OPERATIONS_ROLE` into two roles and gating the `linkLegacyValidatorIds` by a role owned by a multisig.

Customer Response:

Fixed in [5daa413fda](#)

Fix review:

Fix confirmed

Informational Issues

I-01. Pod-less exit requests can be accepted before validator activation

Description: A pubkey is linked to its `EtherFiNode` immediately when the initial 1 ETH deposit is submitted. For pod-less withdrawal requests, this mapping is the only evidence used to authorize the source pubkey.

Unlike the EigenPod path, which requires the validator to have been proven and marked active, the direct EIP-7002 path does not establish that the validator has received its remaining funding or entered the active validator set. Consequently, a request submitted during the period between the initial deposit and activation succeeds on the execution layer but is later discarded by consensus processing.

The transaction still pays the EIP-7002 fee, consumes `EXIT_REQUEST_LIMIT_ID` capacity, and emits `ValidatorWithdrawalRequestSent` and `ExitRequested`. Automation may therefore believe an exit is underway when no exit will occur, potentially delaying emergency exits until the request is noticed, resubmitted, and rate-limit capacity becomes available again.

Recommendation: Since this behaviour is under only `ExecutorOperations` access control, consider acknowledging and documenting this behaviour.

Customer's response: Acknowledged

I-02. Dropping the `getEigenPod() != address(0)` precondition makes `deployedEtherFiNodes` the sole trust anchor for withdrawal credentials, and `backfillExistingEtherFiNodes` performs no verification

Description: Before this diff, `registerBeaconValidators` required `address(IEtherFiNode(etherFiNode).getEigenPod()) != address(0)`. That check was the only on-chain evidence that the address passed as `etherFiNode` was a genuine, protocol-controlled `EtherFiNode`. The diff removes that line and replaces the credential derivation with `withdrawalCredentialTarget(etherFiNode)`, whose only gate is `_validateNode` leading to `stakingManager.deployedEtherFiNodes(node)`. `deployedEtherFiNodes` has two writers: `instantiateEtherFiNode` and `backfillExistingEtherFiNodes(address[] calldata nodes)`. The latter is `only0OperatingMultisig` and does no verification whatsoever. Consequence: a single wrong, typo'd, or stale entry in a backfill list now becomes a permanently valid withdrawal-credential target. `withdrawalCredentialTarget` will return that address verbatim as soon as its `getEigenPod()` read returns zero. `registerBeaconValidators` and `createBeaconValidators` will accept it, and `LiquidityPool` will send ETH to the beacon chain with `0x02 + <that address>` credentials. Because withdrawal credentials are immutable and the only ETH-recovery paths require the target to be a real `EtherFiNode` implementation, every wei exiting those validators is permanently unrecoverable.

Recommendation: Have `backfillExistingEtherFiNodes` (and `_validateNode`) assert that the address is a beacon proxy of `etherFiNodeBeacon`, e.g. by reading the ERC-1967 beacon slot from the candidate address and requiring it to equal `address(etherFiNodeBeacon)`, or by requiring a successful `IEtherFiNode(node).etherFiNodesManager() == address(this)` round-trip. Move `backfillExistingEtherFiNodes` behind the upgrade/operating timelock. Alternatively, consider acknowledging and documenting this behaviour.

Customer's response: Fixed in [3e8db924bb](#)

Fix review: Partially fixed. A malicious node, which returns the correct `etherFiNodesManager` can still bypass the `_validateNode` checks.



I-03. `sweepFunds(address)` performs an unvalidated external call and can emit forged `FundsTransferred` events

Description: `sweepFunds` was changed from `uint256 id` (resolved through `etherfiNodeAddress`) to a raw `address node` with an explicit *"unvalidated"* comment, unlike every other node-taking function in the contract, which calls `_validateNode`. A housekeeping key can therefore make `EtherFiNodesManager` call `sweepFunds()` on any address, including an attacker-deployed contract that returns an arbitrary `balance`, producing `FundsTransferred(node, balance)` events that never corresponded to a transfer. Off-chain accounting/monitoring that consumes `FundsTransferred` can be poisoned, and the manager's identity is lent to an arbitrary callee.

Recommendation: Validate the node (`_validateNode(node)`), and backfill legacy nodes into `deployedEtherFiNodes` rather than dropping the check — every other function in the file already relies on that mapping.

Customer's response: Acknowledged



I-04. Overpaid EIP-7002/7251 fees are never refunded and distort LiquidityPool out-of-LP accounting when swept and are not refunded to the payer

Description: `EtherFiNodesManager` forwards the *entire* `msg.value` to the node, while the pod-less path spends only `fee * requests.length` at the predeploy. Any surplus is left on the node and eventually swept by `_sweepToLiquidityPool().LiquidityPool.receive()` then executes accounting for ETH that was never accounted as out-of-LP, so `totalValueOutOfLp` becomes understated relative to actual beacon-chain/pod holdings while total TVL stays flat. Repeated at scale this both mis-states the out-of-LP figure the oracle rebases against and can make later legitimate sweeps revert on underflow. Any excess above the `totalValueOutOfLp` cap is left stranded on the node. Additionally, the excess value is not refunded to the executor.

Recommendation: Consider refunding the fees to the original caller.

Customer's response: Acknowledged

I-05. NatSpec blocks orphaned by the new insertions now document the wrong functions, including the credential resolver

Locations:

- [/EtherFiNodesManager.sol#L606-L619](#)
- [/EtherFiNode.sol#L305-L316](#)

Description: Both new blocks were inserted between an existing docblock and the function it described, so the docblocks are now attached to the wrong symbols:

- In `EtherFiNodesManager`, the docblock for `addressToWithdrawalCredentials` is now the *first* NatSpec preceding `withdrawalCredentialTarget(address node)`. Solidity/`forge doc`/ABI metadata will attribute it to the credential resolver, and it documents a non-existent `addr` parameter. `addressToWithdrawalCredentials` is left completely undocumented.
- In `EtherFiNode`, the docblock for `_sweepToLiquidityPool` now precedes `disablePod()`, which returns nothing and does not touch the liquidity pool. `_sweepToLiquidityPool` is left undocumented.

This is a correctness problem rather than pure cosmetics because `withdrawalCredentialTarget` is a security-critical resolver whose semantics the rest of the design leans on, and generated documentation/interface metadata will describe it incorrectly.

Recommendation: Consider moving the blocks to their related functions.

Customer's response: Fixed in [ad7f7f862](#)

Fix review: Fix confirmed.

I-06. `srcPubkey == targetPubkey` consolidations against pod-less nodes are guaranteed consensus-layer no-ops, are exempt from the consolidation rate limiter, and still emit success events

Description: `_getTotalConsolidationGwei` deliberately counts zero gwei for requests where `srcPubkey == targetPubkey`, meaning they consume no budget from the `CONSOLIDATION_REQUEST_LIMIT_ID` rate limiter. For pod-less nodes, EIP-7251's `is_valid_switch_to_compounding_request` requires the source to hold Ox01 credentials. A `src == target` request from a pod-less node (which holds Ox02 credentials) fails the switch path and hits an early return, meaning it is dropped by the consensus layer. However, the on-chain path reports full success: it burns the per-request fee at the predeploy and emits `SwitchToCompoundingRequested` and `ValidatorSwitchToCompoundingRequested` events. Because the rate limiter is bypassed and the pod-less `MixedNodeRequest` guard only checks the source's node linkage, an `EXECUTOR_OPERATIONS_ROLE` holder can submit an unbounded number of such requests, inflating the global EIP-7251 excess-request counter and increasing consolidation fees paid by legitimate operations with no protocol-side throttle.

Recommendation: In `EtherFiNode.requestConsolidation`'s pod-less branch, reject `srcPubkeyHash == targetPubkeyHash` (revert rather than emit), since a pod-less validator is compounding by construction and the request can never take effect.

Customer's response: Acknowledged

I-07. NatSpec on the EL-trigger functions now materially misdescribes behaviour for two of the three credential regimes, including the fee and grouping contracts operators depend on

Description:

Locations:

- [/EtherFiNodesManager.sol#L224-L233](#)
- [/EtherFiNodesManager.sol#L280-L289](#)

The diff rewrote the bodies of `requestExecutionLayerTriggeredWithdrawal` and `requestConsolidation` to cover three distinct regimes (pod-less, live pod, retired pod) but only updated the `@dev Access:` lines. The remaining NatSpec is now wrong in ways that matter to the privileged operator who has to hand-build these calls:

- `@notice Triggers EIP-7002 withdrawal requests, grouping by EigenPod automatically.` — no grouping is performed by either implementation; the new `MixedNodeRequest` guard *rejects* batches rather than grouping them, and for a live pod no grouping or checking happens at all.
- `@dev associated etherFiNode is derived from pubkey in the request. Caller should ensure all provided validators share the same eigenpod` — for a pod-less node there is no EigenPod at all; the correct requirement is that all sources share the same `EtherFiNode`.
- `@dev EigenLayer validates that validators belong to the pod automatically.` (consolidation) — false for the pod-less regime, and the diff's own inline comment states it is also false for a retired pod ("`Disabled pod: EL v1.14 stops enforcing membership`", `src/staking/EtherFiNodesManager.sol:254-256`). This is the single most safety-relevant sentence in the block, and it is the one that is now most wrong.

Recommendation: Rewrite both NatSpec to properly reflect the new behaviour.

Customer's response: Fixed in [7fd8cfad4](#)

Fix review: Fix confirmed.



I-08. Exit rate limit uses requested gwei, not actual withdrawal

Description: When someone triggers an exit, the rate limiter is debited using the gwei values in the request itself — either the explicit `amountGwei` for partial withdrawals, or a flat `FULL_EXIT_GWEI` (2,048 ETH) for full exits.

The problem is that the beacon chain doesn't necessarily withdraw that much. For partial exits, consensus caps the payout at the validator's real excess balance: $\min(\text{balance} - 32 \text{ ETH} - \text{pending}, \text{amount})$. So you can request 100 ETH, get 5 ETH, and still burn 100 ETH of rate limit.

A compromised or misbehaving `EXECUTOR_OPERATIONS` key could exploit this by submitting inflated partial amounts on validators with little excess balance, draining the exit bucket without moving equivalent ETH. Full exits have the same mismatch in reverse — they always cost 2,048 ETH of limit even if the validator is smaller.

This is griefing by a trusted role, not theft. Impact is limited in practice: the bucket holds ~61,440 ETH and refills in about a day, and the executor role is already privileged.

Recommendation: Consider acknowledging and documenting this issue.

Customer's response: Acknowledged



Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.